Kinesis

Simulated mobile-robot path tracking with Pure Pursuit and SAC

Ankit Prajapati | 28 October 2026

Study question and measured outcome

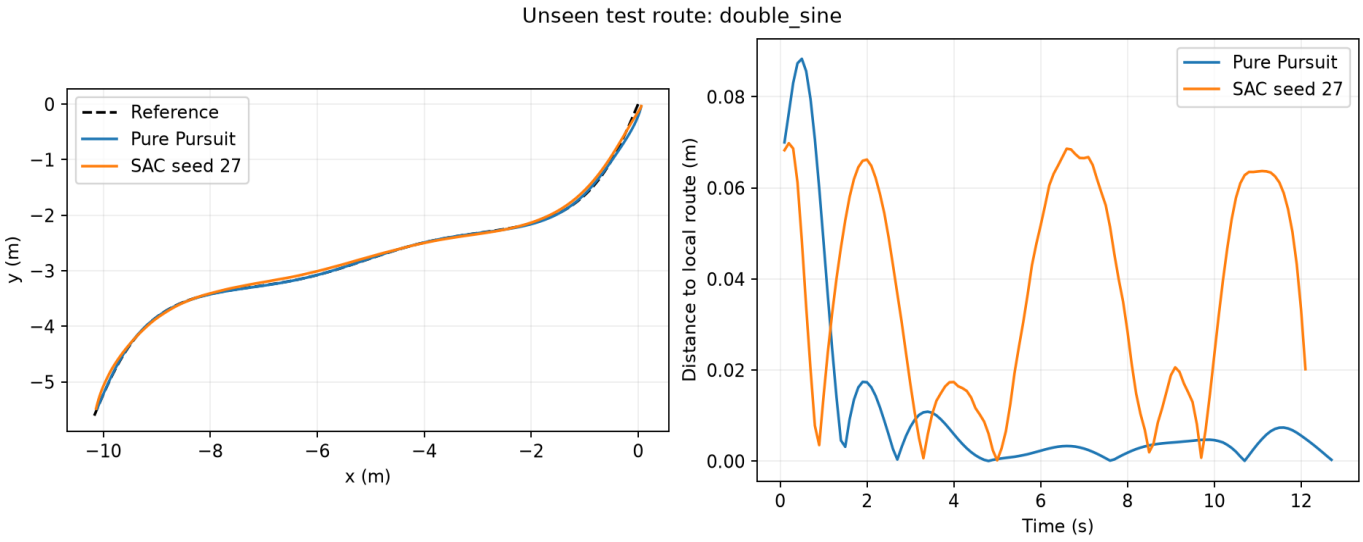
Can a learned continuous-action controller add value beyond a conventional tracker under shared constraints? Both approaches completed all 24 nominal test routes. **Pure Pursuit had lower tracking error, command slew and inference cost.** A command-delay test exposed SAC failures.

Simulator and controller integration

Component	Implementation
Motion	Planar unicycle kinematics; first-order linear/turn-speed response.
Commands	Forward speed 0-1.2 m/s; turn rate -2 to 2 rad/s.
Timing	0.1 s control interval; five integration substeps; response constants 0.25/0.18 s.
Inputs	Same 25-value observation, bounded actions and time allowance for both controllers.
Completion	Ordered gates every 0.5 m, tolerance 0.35 m, plus endpoint/progress requirements.

Evaluation protocol

24 training, 12 validation and 24 test scenarios. Test cases include new initial conditions and route parameters, plus six held-out double-sine paths. Four candidate settings per method use the same validation scenarios; computational tuning costs differ. SAC uses four 12,000-step pilots, then independent seeds 7, 17 and 27 for 80,000 steps each. Selection is frozen before testing.



Fixed unseen route 003; SAC seed 27 selected on validation for illustration. All three seeds are retained in reported results.

Verification: 26 analytic, API and boundary checks pass. A fresh virtual environment reproduces the recorded dependency versions.

Measured results

Paired held-out scenarios | nominal operation and separate disturbances

Nominal tracking

Each model uses the same 24 routes. SAC values are mean +/- sample standard deviation across three training seeds, not confidence intervals.

Metric	Pure Pursuit	SAC, three seeds
Success (%)	100.0	100.0 +/- 0.0
Tracking RMSE (m)	0.043	0.113 +/- 0.066
Elapsed time (s)	12.65	12.24 +/- 1.02
Linear command slew RMS (m/s^2)	0.88	1.23 +/- 0.10
Angular command slew RMS (rad/s^2)	0.59	5.91 +/- 2.18
Mean episode median inference (ms)	0.032	0.390 +/- 0.058

All nominal attempts succeeded, so the elapsed-time comparison uses completed routes. Individual SAC means were 13.33, 11.33 and 12.05 s versus 12.65 s for Pure Pursuit. Faster runs also had larger error and command slew.

Stress tests

Condition	PP success	SAC success	PP RMSE	SAC RMSE
Nominal	100.0%	100.0 +/- 0.0%	0.043 m	0.113 m
Observation noise	100.0%	100.0 +/- 0.0%	0.046 m	0.115 m
200 ms delay	100.0%	90.3 +/- 8.7%	0.059 m	0.175 m
2.5x response times	100.0%	100.0 +/- 0.0%	0.054 m	0.149 m

Noise: 3 cm Gaussian position noise per coordinate and 3 degree heading noise. Disturbances are applied separately. Delay history is observed; nominal training does not expose nonzero delay.

What the comparison establishes

Pure Pursuit completed all 96 attempts across the four conditions. SAC completed 65 of 72 delayed attempts: six missed-gate failures and one time limit. Every model completed the six nominal double-sine cases, but that sample is small. Overall, the conventional baseline is the stronger default for this simulator and training budget.

Error and time summaries include early failures outside the nominal condition. Low error on a short failed attempt does not imply better control. Full per-episode and per-seed records remain available.

Reproducibility and limits

Evidence, scope and next experiments

What is implemented

The project implements the simulator, reference geometry, Gymnasium environment, Pure Pursuit, controller integration, reward/completion safeguards, training pipeline and evaluation. **SAC itself is supplied by Stable-Baselines3 and PyTorch.** The representative policy and both other final seeds are archived with model hashes.

Environment and recorded resources

Python 3.12.11; CPU-only PyTorch 2.14.1; Stable-Baselines3 2.9.2; Gymnasium 1.4.0. The cloud machine has two vCPUs and about 8 GiB RAM on an AMD EPYC 7763 host. One PyTorch thread is used. Final learning calls total 11.13 minutes across three seeds, excluding pilots, evaluation, setup and documentation. No GPU was used.

```
Training source checkpoint: 0df854a866d30330ba32dad38c29c9d8410ea1cc
```

Repeat the checks and inspect a controller

```
bash .devcontainer/setup.sh
```

```
source .venv/bin/activate
```

```
python -m pytest
```

```
python -m kinesis.demo --controller sac
```

To rerun training without overwriting the recorded study, use the pipeline command in README.md with a separate output directory. Hardware and numerical-library differences can affect results; package pinning does not guarantee bitwise identical training.

Interpretation boundaries

- Three training seeds and six cases per route family provide limited evidence about variability and generalization. No significance claim is made.

- The model omits wheel mechanics, slip, obstacles, collisions, terrain, physical localization and real-world safety validation. It is not a motorcycle model.

- Latency is CPU wall time around prediction. It does not establish embedded worst-case timing, memory fit or deployment readiness.

- Noise, delay and response changes are separate tests. Combined disturbances and deployment hardware are future work.

Next experiments

Train with randomized command delays and response parameters, broaden scenario coverage and repeat with more seeds. Compare checkpoint selection on validation under a predeclared rule. A later ROS2 or physical-robot extension needs separate modelling and validation.

Evidence and references

Repository and reproducible source; artifacts/protocol.json, test_episodes.csv, test_results.json, model hashes and training records. The learning report explains each stage and includes interview revision.

Stable-Baselines3 SAC documentation
Coulter (1992), Implementation of the Pure Pursuit Path Tracking Algorithm